

НАЦИОНАЛНА ОЛИМПИАДА ПО ИНФОРМАЦИОННИ ТЕХНОЛОГИИ

ПРОЕКТ 765

ТЕМА:



LeafMap Insights

НАПРАВЛЕНИЕ: РАЗПРЕДЕЛЕНИ ПРИЛОЖЕНИЯ

АВТОР 1:

Тихомир Георгиев Петков

e-mail: tihomir.petkov484@edu.mon.bg

ЕГН: 0741034484

тел: 0895239888

клас: XII

ПГ „Ген. Владимир Заимов”

гр.Сопот

АВТОР 2:

Мариан Петров Балев

e-mail: marian.balev400@edu.mon.bg

ЕГН: 0751114400

тел: 0876607889

клас: XII

ПГ „Ген. Владимир Заимов”

гр.Сопот

РЪКОВОДИТЕЛ:

Тоня Христова Белезирева

телефон: 0893542240

e-mail: tonya.belezireva@edu.mon.bg

длъжност: старши учител практическо обучение

Професионална гимназия „Ген. Владимир Заимов” – гр.Сопот

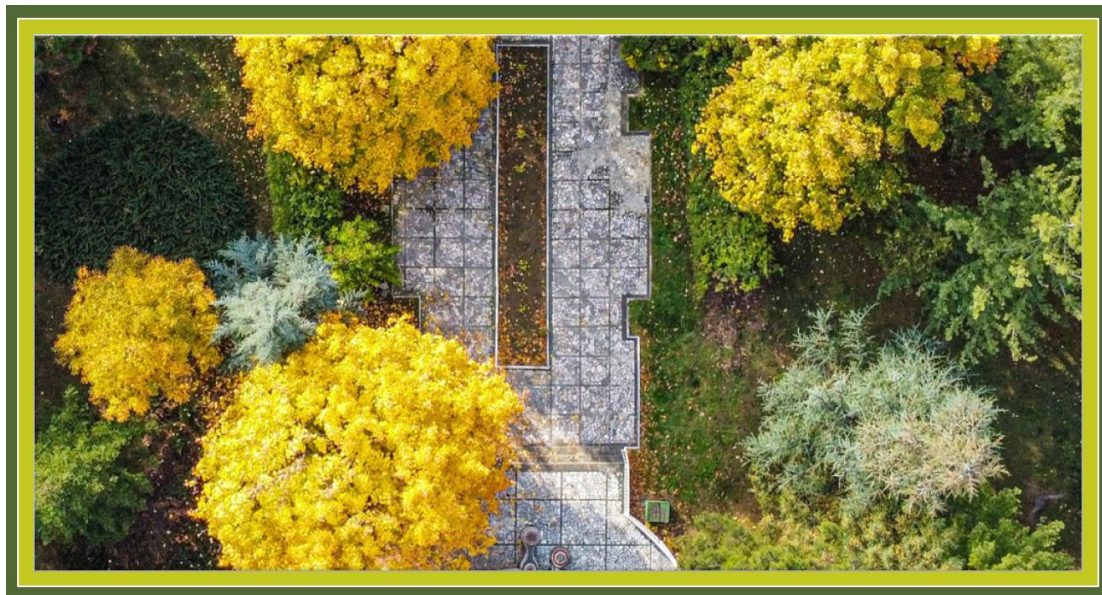
2026 година

СЪДЪРЖАНИЕ:

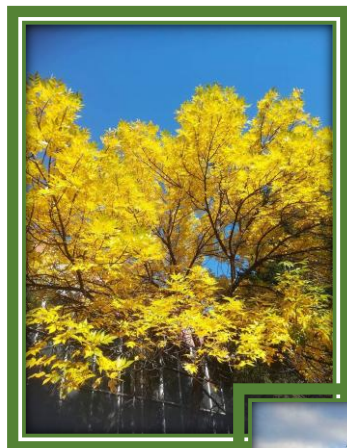
Резюме	3
1. Цели	4
2. Основни етапи в реализирането на проекта	6
3. Ниво на сложност на проекта	8
4. Логическо и функционално описание на решението	9
5. Реализация	15
6. Описание на приложението	18
7. Заключение	19
8. Източници	20

РЕЗЮМЕ

Ландшафтна архитектура е проектирането на обществени места на открито, паркове и градини, както и други структури за постигане на екологични, социално-поведенчески, или естетични резултати.



Устойчивото развитие на съвременния туризъм е неразделно свързан с екологията и биоразнообразието. Паркът на ПГ “Генерал Владимир Заимов” е пример за устойчива еко-система – създадена, пазена и развивана, както нашето училище се грижи за учениците!



1. Цели

Ние, учениците на „Заимов“, прекарваме значителна част от свободното си време в училищния парк. Създаден през 80-те години на миналия век, паркът е неразделна част от училищния ни живот и се поддържа с грижа и внимание. Той е място за отдых, срещи и вдъхновение, мотивира ни да го опознаем и съхраним. Създава уют, хармония и усещане за близост с природата, което ни вдъхнови да създадем дигитално решение за неговото опознаване и съхраняване, като същевременно използваме технологии за образователни цели и насърчаваме екологичното съзнание сред учениците.

LeafMap Insights е разпределена информационна система за каталогизиране и управление на дървета с географско позициониране и ботаническа таксонометрия. „LeafMap Insights“ символизира нашето желание да наблюдаваме, разбираме и съхраняваме природата около нас чрез цифрови инструменти, превръщайки всяко дърво в източник на знание и вдъхновение. Приложима е за всякакви паркови и други еко-системи.

Основната предпоставка за нейната приложимост е масовото използване и лесен достъп за всеки краен потребител. С тази насоченост идеята от учебен STEM проект прерасна в разпределена информационна система „LeafMap Insights“ с Web клиент и отделно мобилно приложение. За нивото на сигурност и правилно разпределяне на ресурсите се създадоха API приложения за поддръжка на БД и управление на потребителите.

Основни акценти и цели:

1.1. Централизирано управление на данни – само Backend API работи с базата данни. Всички операции (CRUD) минават през API, което осигурява консистентност, намалява дублирането и улеснява поддръжката. Web и Mobile приложенията са клиенти без собствена БД, което опростява архитектурата и позволява лесно добавяне на нови клиенти без промени в базата.

1.2. Многоплатформен достъп – системата предоставя достъп през четири приложения:

- Backend API (ASP.NET Core Web API) — централен сървър с база данни и REST endpoints
- Backend API (ASP.NET Core Web API) — централен сървър за управление на потребителски достъп
- Web приложение (ASP.NET Core MVC) — за десктоп браузъри
- Mobile приложение (MAUI) — за мобилни устройства

Всички използват един и същ API и данни, което гарантира синхронизация и единно потребителско изживяване навсякъде.

1.3. Ботаническа таксономия – системата поддържа йерархична ботаническа **таксономия**: Division (отдел) → TaxonomyClass (клас) → Family (семейство) → Genus (род) → Species (вид). Всяко дърво се свързва с таксономични нива, което позволява систематизирано класифициране, търсене и анализ по научни категории.

1.4. Географско каталогизиране – всяко дърво има GPS координати (Latitude, Longitude), което позволява: визуализация на интерактивна карта, филтриране по географски граници (WithinBounds), анализ на разпределение и популации, интеграция с географски информационни системи (GIS)

1.5. Сигурност и автентикация – системата използва JWT (JSON Web Tokens) за автентикация за безопасно управление на потребителски сесии, разделение на права между регистрирани потребители и администратори със съответната защита и автентикация, стандартизиран подход за Web и Mobile клиенти

1.6. Мащабируемост и поддръжка – архитектурата позволява:

- Лесно добавяне на нови клиентски приложения без промени в базата
- Независимо развитие и разгръщане на компонентите
- Използване на стандартни REST API протоколи за интеграция
- Възможност за разширяване с нови функционалности

1.7. Потребителски опит – системата цели да предостави:

- Интуитивен интерфейс за преглед и управление на дървета
- Бърз достъп до информация чрез QR кодове
- Визуализация на данни на интерактивни карти
- Ефективно търсене и филтриране по различни критерии
- Удобство при използване на различни устройства

1.8. Научна и образователна стойност

Чрез систематизираното съхраняване на таксономични данни системата подпомага:

- Научни изследвания в областта на ботаниката и екологията
- Образователни програми за изучаване на растителни видове
- Публично разпространение на знания за местната флора
- Мониторинг и анализ на популациите на дърветата

1.9. Технологична модерност – системата използва съвременни технологии и практики:

- ASP.NET Core за високопроизводителни уеб приложения
- Entity Framework за ORM и управление на база данни
- JWT за безпроблемна автентикация

- MAUI за крос-платформено мобилно развитие
- RESTful API архитектура за стандартизирана комуникация

Тези цели правят LeafMap Insights комплексна система за управление на дървета, която комбинира научна точност, технологична модерност и практическа приложимост за различни потребителски групи и сценарии на използване.

2. Основни етапи в реализирането на проекта

При реализирането на проекта преминах през следните етапи:

2.1. АНАЛИЗ И ПЛАНИРАНЕ

2.1.1. Анализ на изискванията – Функционалните изисквания включват: каталогизиране на дървета с GPS координати, ботаническа таксономия (Family → Genus → Species), CRUD операции за всички таблици, автентикация (Login/Register), роли (регистриран потребител, администратор), визуализация на карта и филтриране по географски граници. Нефункционалните включват: достъп от Web и Mobile, един източник на данни (API), сигурност чрез JWT и възможност за разгръщане на собствен сървър.

2.1.2. Анализ на целевите потребители – Определят се потребителските групи и техните нужди. Регистрираните потребители трябва да преглеждат дървета, да добавят нови и да виждат таксономия. Администраторите трябва да управляват Family, Genus, Species и потребители. Системата трябва да работи както на десктоп (браузър), така и на мобилни устройства.

2.1.3. Архитектурно планиране – Проектира се архитектурата на системата. Приема се модел „един API – много клиенти“: API е единственият слой с достъп до БД; Web и Mobile са тънки клиенти без собствена БД. Определя се комуникацията по HTTP/HTTPS и JSON, както и начините за съхранение на JWT (Session за Web, SecureStorage за Mobile).

2.1.4. Избор на технологии – Избират се технологиите за всеки компонент. За Backend: ASP.NET Core Web API, Entity Framework Core, SQL Server, ASP.NET Core Identity, JWT Bearer. За Web: ASP.NET Core MVC, Razor Views, IHttpClientFactory. За Mobile: .NET MAUI. Критериите са съвместимост, поддръжка, възможност за екип с .NET опит и възможност за общ API договор.

2.1.5. Планиране на базата данни – Проектира се ER диаграмата за БД. Дефинират се таблиците за таксономия (Division, TaxonomyClass, Family, Genus, Species) и централната таблица Tree с полета за GPS, възраст, височина, състояние, QR код и FK към таксономичните нива. Определят се релациите, ограниченията и политиката при изтриване (Restrict). Планират се Identity таблиците за потребители и роли.

2.1.6. Планиране на API endpoints – Описват се REST endpoints, определят се форматите на заявките и отговорите, параметрите и правилата за авторизация (GET без токен, POST/PUT/DELETE с JWT).

2.1.7. Планиране на работата и разпределяне на отговорностите – оценяват се необходимите ресурси и обем на работа за всеки етап. Определят се приоритети и последователност: първо API и БД /автор1/, после Web клиент /автор 2/, после Mobile /двамата автори/, накрая интеграция и дизайн /двамата автори/.

2.2. ИЗБОР И ПОДГОТОВКА НА ХАРДУЕР

Изборът и подготвянето на хардуер са важни за стабилната работа на LeafMap Insights и включват сървърна инфраструктура, машини за разработка и устройства за тестване.

2.3. РЕАЛИЗАЦИЯ

2.3.1. Реализация на back-end компоненти:

- Създаване на бази данни чрез Entity Framework ORM Code-First
- Създаване на API-тата за работа с БД
- Създаване на Services за интеграция на JWT токените
- Създаване на Services за улесняване на работата на controllers
- Създаване на Controllers за обработка на HTTP/HTTPS заявките към API-то

2.3.2. Реализация на front-end компонентите

- Създаване на дизайн и интеграция на карта в Node.js и MAUI приложенияте
- Създаване на валидация и клиентска логика в Node.js приложението
- Създаване на валидация и клиентска логика в MAUI приложението

2.3.3. Тестване на отделните модули

2.4. ИНСТАЛАЦИЯ И КОНФИГУРИРАНЕ НА СЪРВЪРА

Инсталацията и конфигурирането на сървъра са нужни за стабилно хостване на LeafMap Insights API-та и БД. Операционната система, използвана за сървърната част, е Ubuntu Server. За мрежова сигурност е използван Firewall, като се отварят само необходимите портове за ASP API-тата и node.js клиента.

2.5. ИНСТАЛИРАНЕ НА УЕБ ИНТЕРФЕЙСА И МОБИЛНОТО ПРИЛОЖЕНИЕ

Инсталирането на уеб интерфейса (LeafMap Insights Web NodeClient) и неговата интеграция с мрежата включват разгръщане на приложението и настройка на връзката му с API. Уеб приложението е Node.js клиент без собствена база данни. Всички данни идват от Backend API чрез HTTP заявки.

3. Ниво на сложност на проекта

LeafMap Insights е проект със средна сложност. Той комбинира няколко компонента (API, Web, Mobile), но всеки от тях използва познати технологии и стандартни подходи. Сложността идва от:

- **Разпределена архитектура** – три приложения, които трябва да работят заедно
- **Различни клиенти** – Web (браузър) и Mobile (Android) с различни изисквания
- **Автентикация и авторизация** – JWT, роли, защита на endpoints
- **Таксономия и география** – йерархия от обекти и филтриране по координати

Сложността не е в отделните технологии, а в тяхната интеграция и в поддържането на единна логика в различни среди.

3.1. Back-end приложения – API приложения (Auth API и Data API)

Двете API приложения са на **ASP.NET Core 8.0** (.NET 8.0), с Entity Framework Core 8.0.12, Microsoft.IdentityModel.Tokens и System.IdentityModel.Tokens.Jwt 8.2.1, Swashbuckle (Swagger) 6.6.2. Мобилният клиент е на **.NET MAUI 9.0** (net9.0-android, net9.0-ios, net9.0-maccatalyst, net9.0-windows10.0.19041.0), с Microsoft.Extensions.Http 9.0.0. Node.js клиентът използва **Node.js 18+**, **Express 4.18.2** и **dotenv 16.3.1**; сървърът обслужва статични файлове и подава API адресите чрез GET /api-config. Разделянето на автентикацията и данните в два отделни ASP.NET Core 8.0 проекта увеличава сложността по няколко линии. И двата сървъра използват една и съща база данни (общ LeafMapDbContext и connection string), което изисква синхронизирани миграции и еднаква схема. JWT токена се издава само от Auth API, а Data API само го валидира; за да работят заявките, и двата проекта трябва да ползват еднакви стойности за тайния ключ (Jwt:Key), издател (Jwt:Issuer) и аудитория (Jwt:Audience) в appsettings. При несъответствие токена се отхвърля от Data API с 401. Освен това клиентите трябва да знаят два различни базови URL — един за вход/регистрация и един за дървета и таксономия — което усложнява конфигурацията и деплоя.

3.2. Front-end мобилно приложение - MAUI App .NET 9.0

Мобилният клиент трябва да комуникира едновременно с Auth API и с Data API. Това изисква два базови адреса (ApiSettings.BaseUrl и ApiSettings.AuthApiBaseUrl) и превключване между тях — един HttpClient с BaseAddress за Data API и изрични пътища към Auth API при логин/регистрация. JWT се пази в SecureStorage, а ролите — в Preferences или чрез парсане на JWT payload при рестарт; това изисква коректно обработване на claim типа за роля от ASP.NET Core Identity в JWT. При емулятор (напр. Android) localhost не сочи към хост машината, затова

URL-овете трябва да се сменят (напр. `http://10.0.2.2:5202` и `http://10.0.2.2:5203`), което добавя конфигурационна и тестова сложност.

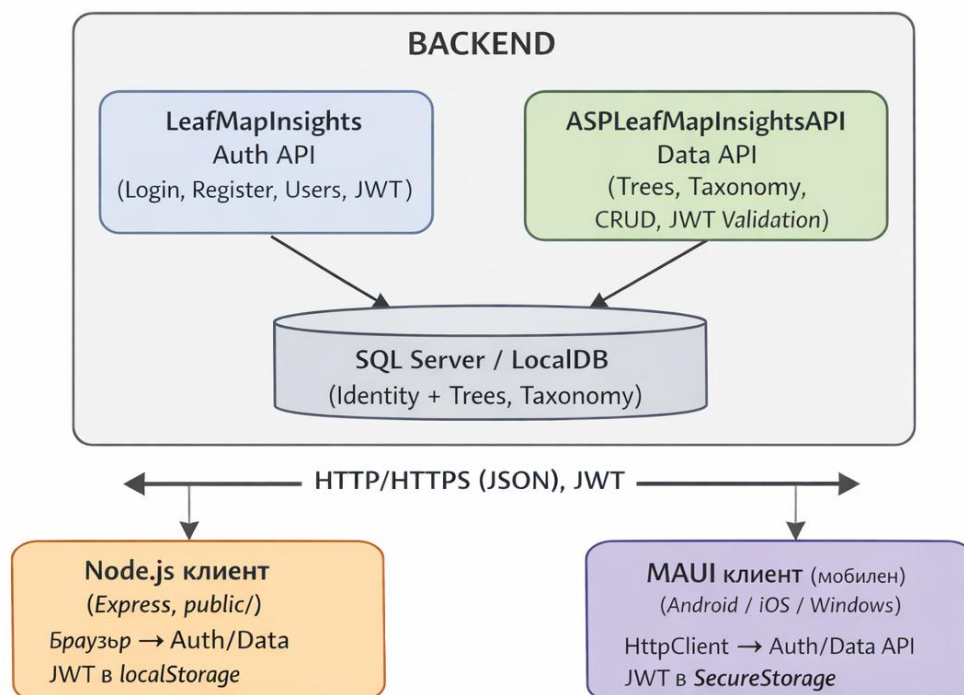
3.3. Front-end уеб приложение - Node.js клиентът (Node 18+, Express 4.x)

Node сървърът не прави прокси заявки — обслужва само статични файлове и подава конфигурация чрез `GET /api-config` (адресите идват от `.env`). Всички заявки се изпращат от брауъра директно към съответното API; в `app.js` трябва да се разграничават двата базови URL и да се използват две отделни функции за заявки — една за Data API и една за Auth API, като и двете добавят един и същ JWT от `localStorage` в Authorization. Това води до дублиране на логиката за заявки и грешки и изисква правилно задаване на `API_BASE_URL` и `AUTH_API_BASE_URL` в средата.

4. Логическо и функционално описание на решението

4.1. Обща архитектура

Системата е разделена на **backend** (две API услуги и обща база данни) и **клиенти** (уеб и мобилен), които комуникират по HTTP/HTTPS с JSON. Единствен източник на данни е сървърната база; клиентите нямат собствена БД и само викат API endpoints.



Фиг. 1 Компонентна схема на LeafMap Insights

4.2. Модули на backend

4.2.1. LeafMap Insights Auth API (Auth API)

- **Роля:** автентикация и управление на потребители; единствено тук се издава JWT.
- **Модули:**
- **Controllers:**
 - AuthController – POST api/auth/login, POST api/auth/register; проверка с Identity, генериране на JWT чрез IJwtService, връщане на { token, email, expiresAt, roles }.
 - UsersController – GET/POST/PUT/DELETE api/users за потребители и роли; достъп само с роля Admin ([Authorize(Roles = "Admin")]).
- **Слой данни:** използва общия LeafMapDbContext (чрез референция към ASPLearnMapInsightsAPI) – същите Identity таблици (AspNetUsers, AspNetRoles и т.н.) и същата база.
- **Услуги:** IJwtService / JwtService – генерира JWT с claims (userId, email, roles), подпис с Jwt:Key, Jwt:Issuer, Jwt:Audience от конфигурация.
- **Конфигурация:** Identity (пароли, роли), JWT параметри, connection string; при старт – миграции и seed на роли User/Admin и администраторски акаунт.
- **Взаимодействия:** Клиентите изпращат POST за login/register; при успех получават JWT и го използват при заявки към Auth API (api/users) и към Data API. Auth API не вика Data API.

4.2.2. ASPLearnMapInsightsAPI (Data API)

- **Роля:** бизнес данни – дървета и таксономия; само валидира JWT (не го издава).
- **Модули:**
- **Data:**
 - LeafMapDbContext – наследява IdentityDbContext<IdentityUser>; съдържа DbSet за Divisions, TaxonomyClasses, Genera, Families, Species, Trees и конфигурация на релации (FK, Restrict при изтриване).
 - SeedData – първоначално запълване на таксономия при празна БД.
 - **Models:** Entity класове (Tree, Division, Species, Genus, Family, TaxonomyClass) и DTO/заявки за Auth (LoginRequest, RegisterRequest, LoginResponse) – ползват се и от Auth API чрез референция към този проект.
- **Controllers (REST, route api/[controller]):**

- **TreesController** – GET всички/по id, GET WithinBounds (по координати за карта), POST/PUT/DELETE с [Authorize].

- **DivisionsController, TaxonomyClassesController, GeneraController, FamiliesController, SpeciesController** – CRUD за референтни данни; записът (POST/PUT/DELETE) защитен с [Authorize].

- **Program/конфигурация:** JWT Bearer валидация със същите Key/Issuer/Audience като Auth API; CORS; миграции при старт.

- **Взаимодействия:** Приема заявки от клиенти с заглавка Authorization: Bearer <token>. Middleware валидира токена; при успех заявката стига до контролера. Не вика Auth API; потребителите и ролите са в същата БД, до която Auth API също има достъп.

4.2.3. Общ слой данни (споделен между двата API)

- **База данни:** една SQL Server/LocalDB инстанция – таблици за Identity (потребители, роли) и за домейна (дървета, таксономия).

- **Синхронизация:** И двата API проекта трябва да използват еднакви connection string и еднакви JWT параметри (Key, Issuer, Audience), за да има единна автентикация. Миграциите са дефинирани в Data API проекта; Auth API ги изпълнява чрез същия LeafMapDbContext.

4.3. Модули на клиентите

4.3.1. Node.js клиент (LeafMapInsightsNodeClient)

- **Роля:** уеб приложение – подава конфигурация и обслужва статични страници; данните минават директно от браузъра към API-тата.

- **Модули:**

- **Сървър (server.js):** Express; express.static('public') за HTML/JS/CSS; един endpoint GET /api-config, който връща { apiBaseUrl, authApiBaseUrl } от .env (API_BASE_URL, AUTH_API_BASE_URL). Не прави прокси заявки към API.

- **Клиентски код (public/app.js):**

- Конфигурация: четене/запис на API адреси в localStorage; при зареждане може да викне /api-config и да ги обнови.

- api(endpoint, options) – заявки към **Data API** (дървета, таксономия); добавя Authorization: Bearer от localStorage.

- `authApi(endpoint, options)` – заявки към **Auth API** (login, register, `api/users`); също добавя Bearer при наличие на токен.

- Функции за UI: проверка на връзка с API, вход/регистрация/изход, списък дървета, детайли за дърво, карта, таксономия, добавяне на дърво, при нужда админ (потребители).

- **Страници (public/.html): *index.html, login.html, register.html, trees.html, tree.html, taxonomy.html, add-tree.html, admin.html*** – всяка зарежда *app.js* и използва общите функции за заявки и рендиране.

- **Взаимодействия:** Браузър → GET `/api-config` → Node връща URL-ите; браузър → директно POST `api/auth/login` или `api/auth/register` към Auth API; браузър → директно GET `api/trees`, `api/divisions` и т.н. към Data API. JWT се пази в `localStorage` и се поднася при всяка заявка към двата API.

4.3.2. MAUI клиент (MAUILeafMapInsights)

- **Роля:** нативно мобилно/десктоп приложение за същите функции като уеб клиента – вход, дървета, карта, таксономия, добавяне на дърво, админ.

- **Модули:**

- **Услуги:**

- `AuthService` – съхранение/четене/изтриване на JWT в `SecureStorage`; запис на роли в `Preferences` или извличане от JWT payload при рестарт; `IsAdminAsync()`, `IsLoggedInAsync()`.

- `LeafMapApiService` – един `HttpClient` с `BaseUrl` за Data API; преди всяка заявка `EnsureTokenAsync()` задава текущия токен от `AuthService` в заглавката; методи за Login/Register към Auth API (отделни URL от `ApiSettings.AuthApiBaseUrl`); методи за `trees`, `divisions`, `taxonomy`, `users` (Admin).

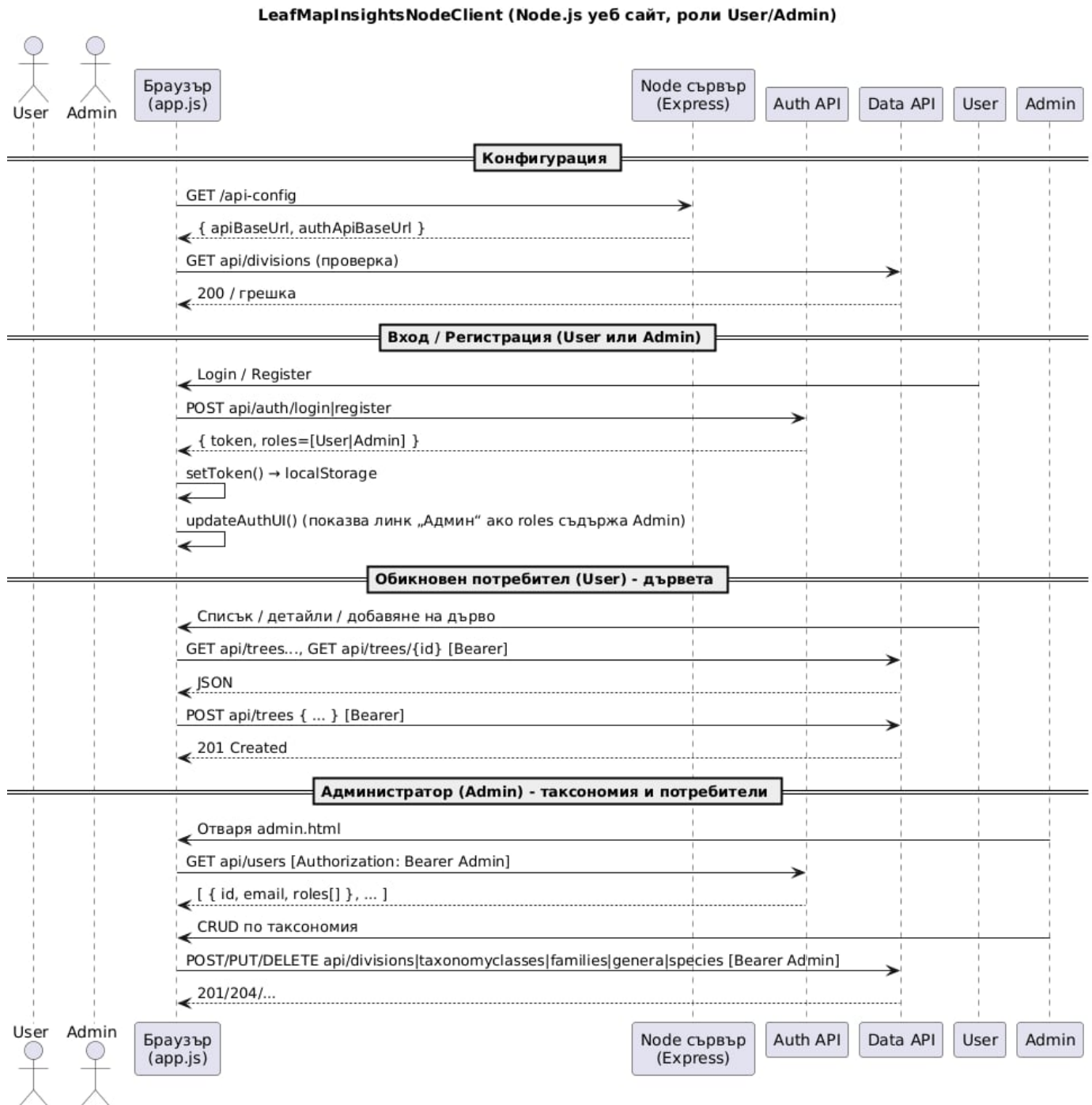
- **Конфигурация:** `ApiSettings` – статични `BaseUrl` (Data API) и `AuthApiBaseUrl` (Auth API).

- **Страници (Pages):** `LoginPage`, `RegisterPage`, `HomePage`, `TreesPage`, `MapPage`, `TaxonomyPage`, `AddTreePage`, `TreeDetailPage`, `AdminPage` – XAML + code-behind; извикват `LeafMapApiService` и `AuthService`, навигират чрез `Shell` (`AppShell`).

- **Навигация (AppShell.xaml):** Flyout с начало, дървета, карта, таксономия, админ (видим само за Admin), вход; допълнителни маршрути за регистрация и детайли/добавяне на дърво.

- **Взаимодействия:** Login/Register → POST към Auth API; всички останали заявки (дървета, таксономия, потребители) → Data API или Auth API (`api/users`) с един и същ JWT в заглавката. Няма директна комуникация между Node и MAUI – само през API-тата.

4.4. Взаимодействия между модулите (поток данни)



Фиг. 2 Потребителски сценарии за модулите в LeafMap Insights

Диаграмата показва **взаимодействията** между участниците в Node.js уеб клиента и двата backend API при **различни роли** (User и Admin).

Участници (lifelines)

- **User / Admin** – потребителите (обикновен потребител и администратор).
- **Браузър (app.js)** – клиентската част на уеб приложението; логиката за заявки и UI е в app.js.
- **Node сървър (Express)** – сървърът на уеб приложението (Node.js + Express).
- **Auth API** – API за автентикация и потребители (.NET).
- **Data API** – API за данни (дървета, таксономия) (.NET).

Конфигурация

- Браузър изпраща **GET /api-config** към **Node сървър**.
- **Node сървър** отговаря с обект { **apiBaseUrl**, **authApiBaseUrl** } – адресите на Data API и Auth API.
- Браузър прави **GET** към **Data API** на **/api/divisions** за проверка на връзката.
- **Data API** отговаря с **200** при успех или с грешка.

Вход / регистрация (User или Admin)

- **User** или **Admin** въвежда данни за вход или регистрация в **браузъра**.
- Браузър изпраща **POST** към **Auth API** на **/api/auth/login** или **/api/auth/register**
- **Auth API** връща JSON с **token** и масив **roles** (напр. ["User"] или ["User", "Admin"]).
- В **браузъра** се изпълняват:
- **setToken()** → **localStorage** – запазване на токена в localStorage.
- **updateAuthUI()** – обновяване на интерфейса; ако в roles има **Admin**, се показва линк „Админ“.

3. Обикновен потребител (User)

- **User** използва **браузъра** за: списък дървета, детайли за дърво, добавяне на дърво.
- За списък/детайли **браузър** изпраща **GET** към **Data API**: **/api/trees** или **/api/trees/{id}** с **Authorization: Bearer /token/**;
- **Data API** връща **JSON** с данните за дърветата.
- За добавяне на дърво **браузър** изпраща **POST** към **Data API** на **/api/trees** с тяло и същия **Bearer** токен.

- **Data API** отговаря с **201 Created**.

4. Администратор (Admin) – управление на таксономия и потребители

- **Admin** отваря **admin.html** в **браузъра**.
- За списък потребители **браузър** изпраща **GET** заявка към **Auth API** на endpoints

/api/users с **Authorization: Bearer /token/** (токенът трябва да е с роля Admin).

- **Auth API** връща списък от потребители във вид **{ id, email, roles[] },**

• За **CRUD** по таксономия **браузър** изпраща **POST, PUT** или **DELETE** заявки към **Data API** на endpoints като **/api/divisions, /api/taxonomyclasses, /api/families, /api/genera, /api/species**, отново с **Bearer** токен с роля Admin.

- **Data API** отговаря с кодове за успех (**201, 204** и т.н.).

5. Реализация – обосновка за използвани технологични средства, алгоритми, литература, програмни приложения и др.

5.1. Технологии и средства

5.1.1. Backend (сървърна част)

За двата API сървъра е използван **ASP.NET Core 8.0** (.NET 8). Платформата предлага вградена поддръжка за **REST API, dependency injection**, конфигурация и крос-платформено изпълнение. **Entity Framework Core 8.0.12** осигурява **ORM** за работа с **SQL Server/LocalDB**, миграции на схемата и заявки чрез **LINQ**. За автентикация и управление на потребители и роли е използван **ASP.NET Core Identity** (**Microsoft.AspNetCore.Identity.EntityFrameworkCore 8.0.12**), който интегрира с **EF Core** и предоставя хеширане на пароли, управление на роли и токени. За издаване и валидиране на токени е използван **JWT** чрез пакетите **Microsoft.IdentityModel.Tokens** и **System.IdentityModel.Tokens.Jwt** (8.2.1), а в **Data API** – **Microsoft.AspNetCore.Authentication.JwtBearer** (8.0.12) за проверка на **Bearer** токен в **middleware**. Документацията на API е генерирана с **Swashbuckle (Swagger) 6.6.2**.

5.1.2. Уеб клиент

За уеб клиента е използван **Node.js** (версия 18+) с **Express 4.18.2** като сървър за обслужване на статични файлове и **endpoint** за конфигурация (**GET /api-config**). Конфигурацията на средата се зарежда чрез **dotenv 16.3.1** от **.env**. Клиентската логика е на чист **JavaScript (ES6+)** в **браузъра**, без допълнителни фреймуърци, за по-лесно поддържане и съвместимост.

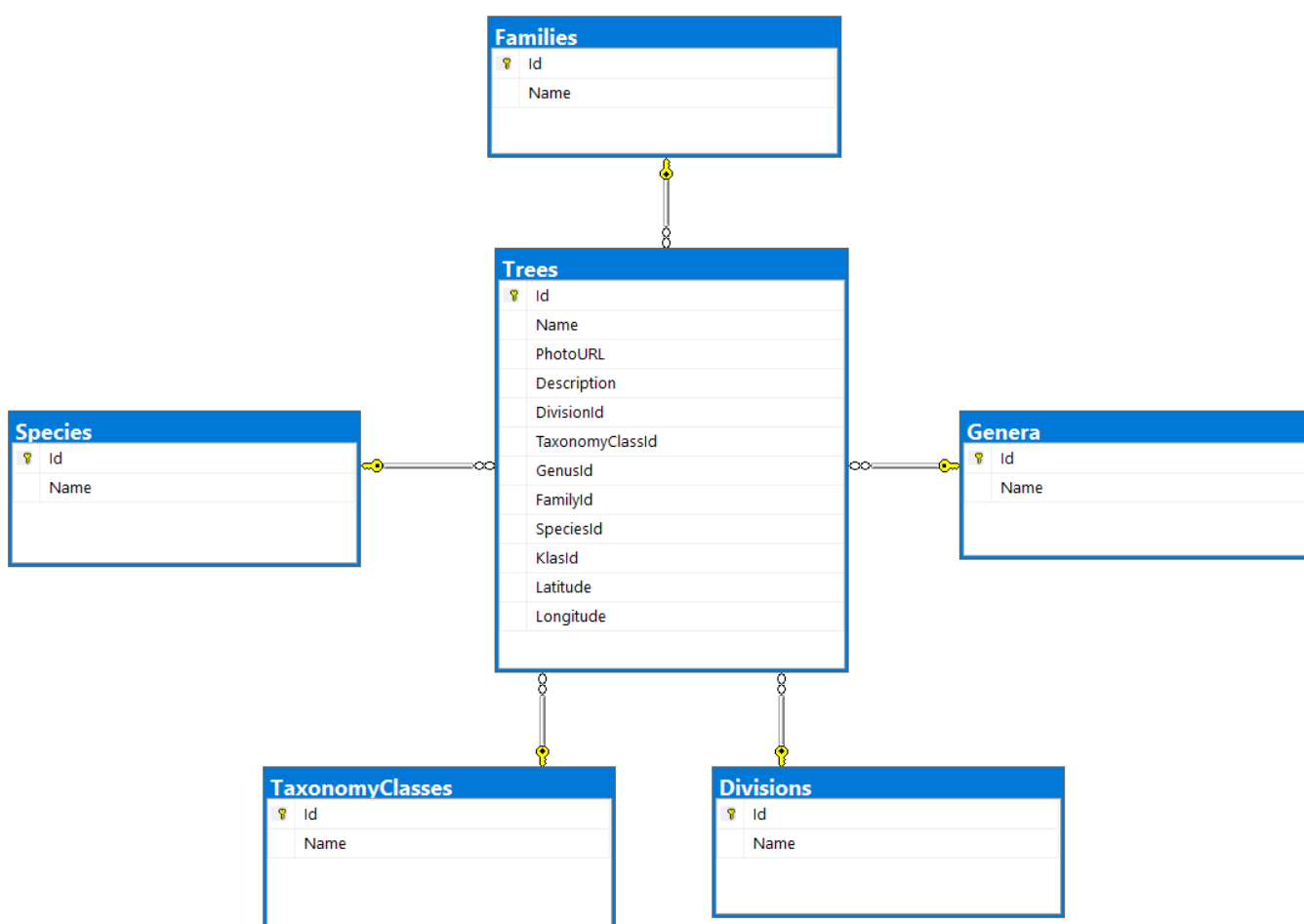
5.1.3. Мобилно и десктоп приложение

За крос-платформеното приложение е използван **.NET MAUI (net9.0)** с целеви платформи **Android, iOS, Mac Catalyst** и **Windows**. Използвани са **Microsoft.Maui.Controls** и

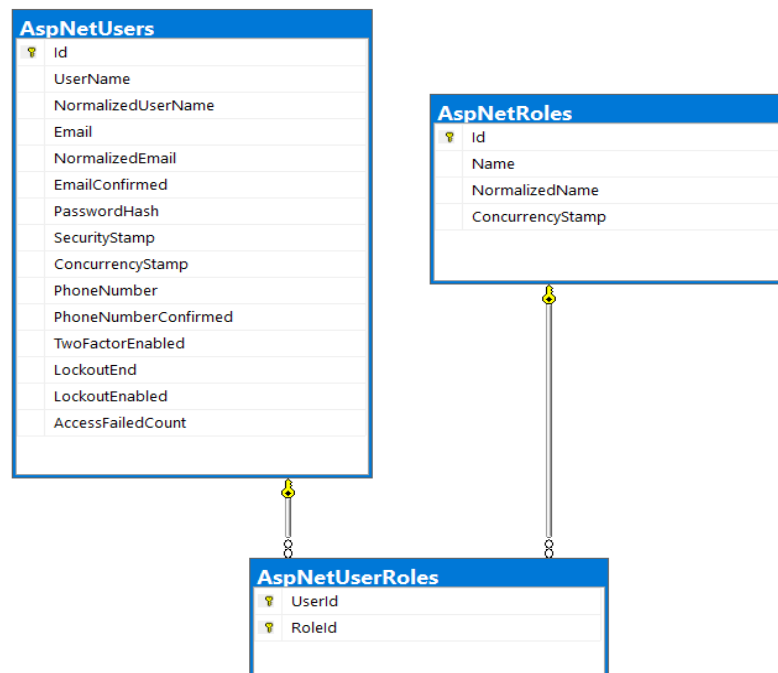
Microsoft.Maui.Controls.Maps за карти, **Microsoft.Extensions.Http 9.0.0** за HTTP заявки. За сигурно съхранение на JWT на устройството се използват платформените API **SecureStorage** и **Preferences**.

5.1.4. База данни

Данните се съхраняват в **SQL Server** (в разработка – LocalDB). Един и същ **LeafMapDbContext** (наследник на **IdentityDbContext**) обслужва и двата API проекта – таблици за Identity (**AspNetUsers**, **AspNetRoles** и др.) и основни таблици с данни (**Trees**, **Divisions**, **Species**, **Genus**, **Family**, **TaxonomyClass**). Това позволява единна схема и една конфигурация на връзката.



Фиг. 3 E/R diagram Основни таблици с данни



Фиг. 4 E/R diagram Данни з потребители - таблици за Identity

5.2. Алгоритми и протоколи

JWT (JSON Web Token)

Токенът се генерира след успешен логин или регистрация в Auth API. Използва се симетричен подпис **HMAC-SHA256** (SecurityAlgorithms.HmacSha256) с таен ключ от конфигурацията (Jwt: Key).

Пароли

ASP.NET Core Identity по подразбиране използва **PBKDF2** с HMAC-SHA256 за хеширане на пароите; пароите не се съхраняват в чист вид. Това е в съответствие с препоръките за съхранение на пароли (напр. NIST, OWASP).

REST и HTTP

Комуникацията между клиенти и API е по **HTTP/HTTPS** с **JSON** в тялото на заявките и отговорите. Ресурсите са организирани по REST конвенция: GET за четене, POST за създаване, PUT за актуализация, DELETE за изтриване. Защитените операции изискват заглавка **Authorization: Bearer /token/**. **CORS** е конфигуриран така, че уеб и мобилни клиенти от различни origin/порт да могат да викат API-тата.

5.3. Помощни програмни приложения и среди

За разработка и тестване са използвани: **Visual Studio 2022** или **Visual Studio Code** с C# разширения за .NET и MAUI проектите; **Node.js** (LTS 18+) и **npm** за Node клиента; **SQL Server LocalDB** или пълна инстанция на SQL Server за базата данни; **draw.io (diagrams.net)** за компонентни и sequence диаграми; **Git** за контрол на версиите. За мобилно тестване – **Android емулатор** и/или физическо устройство; при нужда **Xcode** и емулатор за iOS. Документацията на API се преглежда чрез вграденото **Swagger UI** при стартиране на API проектите в режим Development.

6. Описание на приложението

6.1. Инсталация и стартиране

Системата се състои от два backend API сървъра и един или повече клиента. За пълноценна работа първо се стартират API-тата, след това клиентът (уеб или мобилен).

- **Data API (ASPLoadMapInsightsAPI):** изисква .NET 8 SDK и SQL Server (или LocalDB). В appsettings.json се задава connection string към базата. Стартиране: dotnet run в папката на проекта; по подразбиране слуша на https://localhost:7234 или при зададени URL-и напр. dotnet run --urls "http://0.0.0.0:5202" за достъп от емулатор/други машини.

- **Auth API (LeafMapInsightsAuthAPI):** също .NET 8, използва същата база данни и същите JWT параметри (Jwt:Key, Jwt:Issuer, Jwt:Audience) в конфигурацията. Стартиране: dotnet run в папката на Auth API; при нужда dotnet run --urls "http://0.0.0.0:5203".

- **Node.js уеб клиент (LeafMapInsightsNodeClient):** изисква Node.js 18+. Инсталация: npm install, копиране на .env.example към .env и задаване на API_BASE_URL и AUTH_API_BASE_URL. Стартиране: npm start; отваряне в браузър на http://localhost:3000.

- **Статичен уеб клиент (LeafMapInsightsStaticWeb):** не изисква инсталация; в app.js се задава API_BASE_URL и при двата API – AUTH_API_BASE_URL. Стартиране: отваряне на index.html директно или чрез локален сървър (напр. npx serve .).

- **MAUI приложение (MAUILeafMapInsights):** изисква .NET 9 SDK и Visual Studio / Rider с поддръжка за MAUI. В Services/ApiSettings.cs се задават BaseUrl (Data API) и AuthApiBaseUrl (Auth API). За емулатор/устройство localhost не работи – използва се напр. http://10.0.2.2:5202 и http://10.0.2.2:5203 (Android емулатор) или IP на хоста при реално устройство. Стартиране: избор на Android Emulator и Run (F5) или dotnet run -f net 9.0-android.

6.2. Указания за използване

Потребителят отваря уеб сайта или мобилното приложение. Може да преглежда списък с дървета, детайли, карта и таксономия без вход. За добавяне на дърво и (при Node/MAUI) за админ функции е нужен вход или регистрация – след успешен логин се издава JWT и се пази в браузъра (localStorage) или в приложението (SecureStorage). В уеб интерфейса навигацията е чрез линкове (начало, дървета, карта, таксономия, добави дърво, вход, админ); в MAUI – чрез меню (Flyout) и страници за вход, регистрация, дървета, карта, таксономия, добавяне на дърво и админ панел. Администраторът има достъп до управление на потребители (Auth API) и при нужда до CRUD по таксономия (Data API).

6.3. Поддръжка

Поддръжката включва: актуализация на зависимостите (NuGet за .NET, npm за Node); съобразяване с промени в .NET и Node LTS версии; поддържане на единна конфигурация за JWT и connection string между двата API; проверка на CORS настройките при промяна на домейни/портове на клиентите; документиране на промените в архитектурата и инструкциите за стартиране (напр. HOW_TO_RUN.md, README файловете в съответните проекти). Базата данни се поддържа чрез Entity Framework Core миграции в Data API проекта; при нужда се изпълняват миграции и при Auth API (споделен DbContext).

7. Заключение

Реализирана е модулна система за преглед и управление на данни за дървета и таксономия (LeafMap Insights), с разделение на автентикация (Auth API) и данни (Data API), единна база данни и JWT автентикация. Уеб клиентите (Node.js и статичен HTML/JS) и мобилното/десктоп приложението (MAUI) комуникират с двата API по един и същ договор (REST, JSON), което позволява многоплатформен достъп и ясна роля на всеки компонент. Постигнат е работещ цикъл: регистрация и вход, преглед на дървета и таксономия, добавяне на дърво от логнат потребител, ролево управление (User/Admin) и при нужда админ функции за потребители и таксономия.

Приложението е подходящо за учебни и демонстрационни цели, за локално или вътрешно тестване и за прототипиране на подобни системи. До момента то се използва в рамките на разработката и тестване (локално стартиране на API-тата и клиенти, емулатор и браузър); публично разполагане в production зависи от допълнително настройване на сървъри, домейни и сигурност (HTTPS, тайни за JWT, ограничаване на CORS).

Възможности за развитие и усъвършенстване

- **Функционални:** разширяване на таксономията и филтриране; офлайн режим в мобилното приложение с синхронизация при връзка; известия за нови или променени обекти;

експорт/импорт на данни (напр. CSV, GeoJSON); по-богати права и роли (напр. редактор само за определен регион).

- **Технически:** преминаване на API-тата към HTTPS и подходящо съхранение на тайните (Key Vault или сървърни променливи); ограничаване на CORS до конкретни origin-и; rate limiting и допълнителни мерки срещу злоупотреба; refresh токени за по-дълги сесии; контейнеризация (Docker) и деплой в облак; автоматизирани тестове (unit и integration) и CI/CD.

- **Потребителски опит:** подобрения на картата (мащаб, слоеве, търсене); по-удобни форми за добавяне/редакция; теми на приложението (напр. тъмна и/или светла тема); локализация и достъпност.

С тези насоки системата може да се развива както като самостоятелен инструмент за данни за дървета и таксономия, така и като основа за по-сложни приложения в областта на геоданните и управлението на околната среда.

8. ИЗТОЧНИЦИ – Официална документация

1. Microsoft. *ASP.NET Core документация*. Microsoft Learn.
<https://learn.microsoft.com/aspnet/core/> (последен достъп: февруари 2025).
2. Microsoft. *Entity Framework Core документация*. Microsoft Learn.
<https://learn.microsoft.com/ef/core/> (последен достъп: февруари 2025).
3. Microsoft. *.NET MAUI документация*. Microsoft Learn.
<https://learn.microsoft.com/dotnet/maui/> (последен достъп: февруари 2025).
4. Microsoft. *ASP.NET Core Identity*. Microsoft Learn.
<https://learn.microsoft.com/aspnet/core/security/authentication/identity> (последен достъп: февруари 2025).
5. Microsoft. *JWT Bearer authentication в ASP.NET Core*. Microsoft Learn.
<https://learn.microsoft.com/aspnet/core/security/authentication/jwt-authn> (последен достъп: февруари 2025).
6. JWT.io. *Introduction to JSON Web Tokens*. <https://jwt.io/introduction> (последен достъп: февруари 2025).
7. Microsoft. *CORS в ASP.NET Core*. Microsoft Learn.
<https://learn.microsoft.com/aspnet/core/security/cors> (последен достъп: февруари 2025).
8. Microsoft. *SQL Server документация*. Microsoft Learn. <https://learn.microsoft.com/sql/sql-server/> (последен достъп: февруари 2025).